

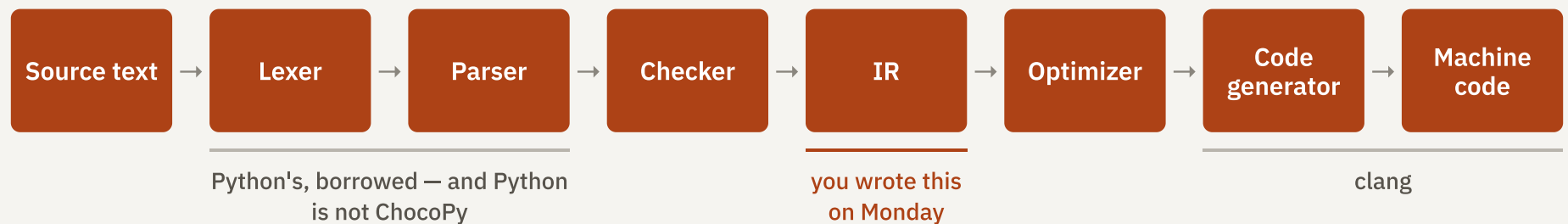
Lecture 4 • Wed 10/7/2026

CS160 Compilers — Fall 2026

Testing Your Compiler.

Yu Feng

Where we are.



Three lectures in, you have a compiler that works. Today is not a new box. Today is the question you have to answer about every box, for the rest of the quarter.

Including the two boxes you didn't write, in a language that isn't the one we're defining. That bill comes due in week 3.

How do you know your compiler is right?

By the end you can say *why* a compiler can't be tested the way you've tested code before.

You can use an interpreter as an answer key — and say what plays that role for the parser, the checker and the optimizer, none of which an interpreter alone can settle.

You can write tests that catch the bug you didn't think of — including the ones a model writes for you.

CS24. You wrote this.

```
def test_sort():  
    assert sort([3, 1, 2]) == [1, 2, 3]  
    assert sort([]) == []  
    assert sort([5, 5]) == [5, 5]
```

You chose an
input.

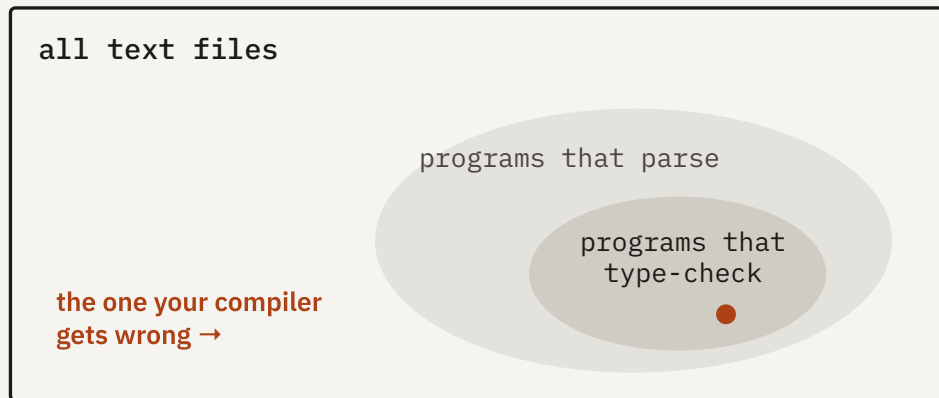
You knew the
answer.

You
compared.

Every one of those three steps breaks when the thing under
test is a compiler. One at a time.

Testing sort	Testing a compiler
Input	a list you typed

Hard thing 1: the input is a program.



To test `sort` you type a list. Any list will do.

To test a compiler you must produce a *program* — and almost no string is one.

The inputs that matter live in a tiny, structured corner of an infinite space, and you have to *construct* them. That is why nobody tests a compiler by typing inputs.

	Testing sort	Testing a compiler
Input	a list you typed	a well-formed program

Hard thing 2: what would you compare it against?

both correct, for the same source

```
%t1 = add i32 3, 4
%t2 = mul i32 3, %t1
```

```
%t1 = add i32 4, 3
%t2 = mul i32 %t1, 3
```

`sort` returns an answer you can compare with `==`.

Your compiler returns *text*, and there is no one right text — these two are both correct.

So you cannot diff against an expected output. You can only compare what the two programs *do*.

“What do I compare the result against?” is called the **test oracle problem**. For a compiler it is the whole difficulty.

	Testing sort	Testing a compiler
Input	a list you typed	a well-formed program
Expected result	a list you wrote down	no unique right answer

Hard thing 3: nothing crashes.

A bug in sort throws, or hands back a visibly wrong list.

A bug in your compiler produces a binary that compiles clean, links clean, runs clean — and prints 3 where it should print 4.

On one program in a thousand.

Lecture 1's average was exactly this, and it is why you did not spot it by reading.

	Testing sort	Testing a compiler
Input	a list you typed	a well-formed program
Expected result	a list you wrote down	no unique right answer
Failure looks like	an exception	a working program with a wrong answer

You can run every line of your compiler and learn nothing.

```
print(1 + -1)
# runs every line of Wednesday's 68-line compiler:
# Constant, BinOp, UnaryOp, the Emitter, compile_program
```

One program, and every line of the 68-line compiler you wrote in Lecture 2 is covered. It says nothing at all about `-7 // 2`.

The bugs are not in the lines. They are in the *assumptions the lines encode*.

Which is also why a model reviewing your code won't find them: it is reading the same lines, holding the same assumption.

	Testing sort	Testing a compiler
Input	a list you typed	a well-formed program
Expected result	a list you wrote down	no unique right answer
Failure looks like	an exception	a working program with a wrong answer
What good tests find	lines you forgot to write	assumptions you didn't know you made

Something has to define the answer. Three candidates.

A document.

The ChocoPy reference manual — thirty-eight pages, and precise about a great deal: §2.5.1 fixes `int` at 32 bits. Now look up `//`. §2.6.3 calls it *division quotient* and stops. It never says which way it rounds.

Rules on paper.

The manual's own inference rules (§5–§6; you'll write the typing half in week 5). The arithmetic rule says the value of `e1 // e2` is `i1 // i2` — precise about everything except the thing you just asked.

A reference interpreter.

A program that computes the answer the slow, obvious way. It has no choice but to pick. And you can run it.

A **specification** is whichever of these you have agreed to be bound by. We use all three this quarter. Today: the only one that is *forced to answer*.

interp.py: the answer key.

```
BIN = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul,
       ast.FloorDiv: operator.floordiv, ast.Mod: operator.mod}
CMP = {ast.Lt: operator.lt, ast.LtE: operator.le, ast.Gt: operator.gt,
       ast.GtE: operator.ge, ast.Eq: operator.eq, ast.NotEq: operator.ne}

def wrap(n):
    # §2.5.1 fixes the range at 32 bits;
    return (n + 2**31) % 2**32 - 2**31    # past it the manual stops, and we decide.

def ev(e, env):
    match e:
        case ast.Constant(value=v): return int(v)
        case ast.Name(id=x): return env[x]
        case ast.BinOp(left=l, op=op, right=r):
            return wrap(BIN[type(op)](ev(l, env), ev(r, env)))
        case ast.UnaryOp(op=ast.USub(), operand=x): return wrap(-ev(x, env))
        case ast.Compare(left=l, ops=[op], comparators=[r]):
            return int(CMP[type(op)](ev(l, env), ev(r, env)))
```

Forty lines — and two of them are decisions the manual never made: `wrap`, and which way `floordiv` rounds. It is slow, and it is *right by definition*: not because it outranks the manual, but because it is the only one of the three that answers every question.

Statements too.

interp.py · the statement half

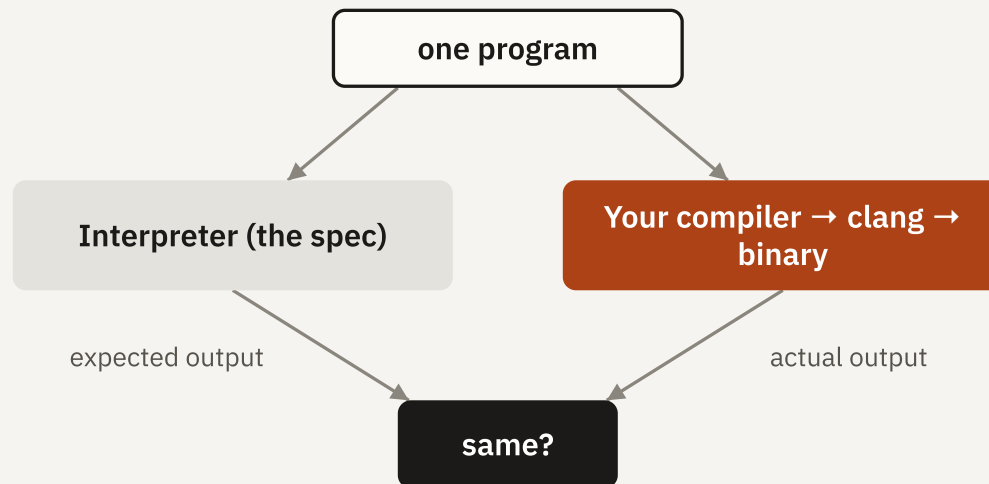
```
def run(s, env, out):
    match s:
        case ast.AnnAssign(target=ast.Name(id=x), value=v):      # x: int = e
            env[x] = ev(v, env)
        case ast.Assign(targets=[ast.Name(id=x)], value=v):      # x = e
            env[x] = ev(v, env)
        case ast.Expr(value=ast.Call(func=ast.Name(id="print"), args=[a])):
            out.append(ev(a, env))
        case ast.If(test=c, body=b, orelse=o):
            for st in (b if ev(c, env) else o): run(st, env, out)
        case ast.While(test=c, body=b):
            while ev(c, env):
                for st in b: run(st, env, out)
```

cocoa3.py · Monday's compiler, the same case

```
case ast.While(test=c, body=body):                                # cocoa3.py
    n = out.fresh_id()
    out.emit(f"br label %cond{n}")
    out.label(f"cond{n}")
    ...
```

Same cases, same shape, walking the same tree. One *does* the loop. The other *writes down instructions that will do it later*.

The oracle.



PA1 has ~20 public tests. The handout supplies 35 tests in `test/pa1/`.

You add three tests in `test/mine/pa1/`.

The interpreter did, in milliseconds.

An **oracle** is anything that tells you whether a test passed. Ours is a comparison: you never have to know the right answer, you only have to have two things that should agree.

Run the same program twice. Compare.

```
$ cat div.py
print(-7 // 2)
print(-7 % 2)
print(7 // -2)
```

```
$ python3 interp.py div.py
```

```
-4
1
-4
```

```
$ python3 -m compiler run div.py
```

```
-3
-1
-3
```

Running two implementations on the same input and comparing is called **differential testing**. Three disagreements. The interpreter is right by definition. The compiler is wrong.

`sdiv` rounds toward zero and `srem` takes the dividend's sign; Python rounds down. **PA1 §5** has the fix — it is in your handout, not on a slide.

Every bug you find this quarter will look like this.

You assumed the target's operation meant the same as the source's.

It meant something slightly different.

The oracle noticed; you didn't.

Slide 8, concretely: this bug is not in a line you forgot to write. It is in a line you wrote confidently.

What counts as “same”?

Same printed output, line for line.

Same *failure*: if the interpreter stops with an error — divide by zero — the binary must exit with an error too, not print garbage.

PA1 §2 requires `cp_error_div()`, the message `Division by zero`, and exit code 2; the handout gives the required error path.

We don't write the programs either.

Slide 5: the inputs are programs, and you can't type enough of them.

So write a program that writes programs.

Generating random inputs and checking what happens is called **fuzzing**.

fuzz.py · trimmed

```
def rand_expr(d=0):
    if d > 2 or random.random() < 0.3:
        return str(random.randint(-9, 9))
    op = random.choice(["+", "-", "*", "/", "%"])
    return f"({rand_expr(d + 1)} {op} {rand_expr(d + 1)})"

def rand_program():
    return "\n".join(f"print({rand_expr()})" for _ in range(3))

for i in range(30):
    p = rand_program()
    want = interpret(p)           # run p under interp.py
    got  = run_compiled(p)       # compile p, run the binary, collect its output
    if want != got:
        print("MISMATCH:\n" + p, want, got); break
```

Random programs, two runs each, compare. Fuzzing plus an oracle is how LLVM itself is tested.

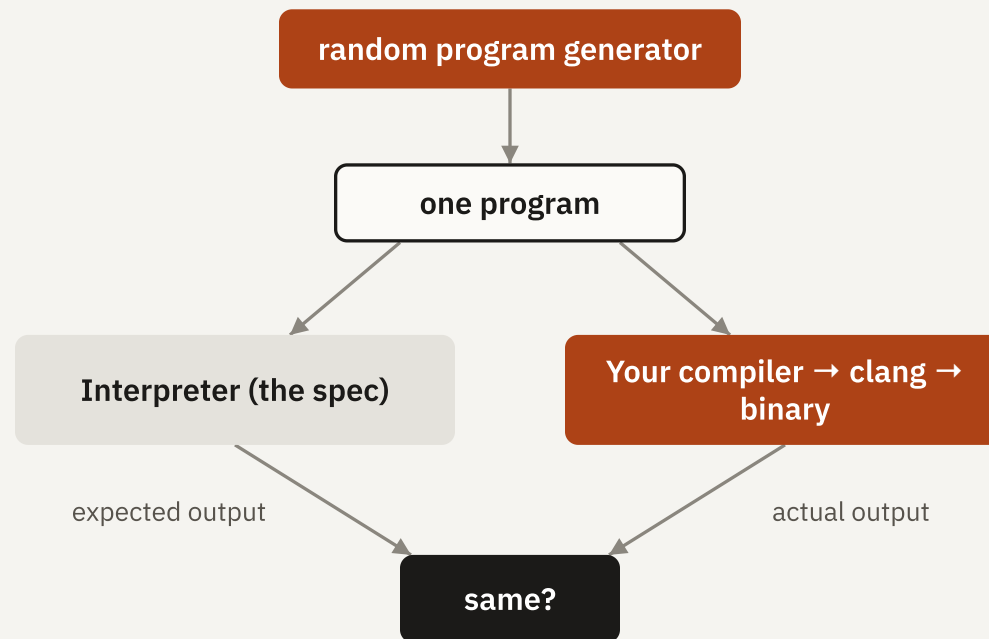
Live · about a minute

It found it on the first try.

```
$ python3 fuzz.py 1
MISMATCH after 1 programs:
print(-7)
print(6)
print(((((-9 + -9) * (-6 - 9)) // ((-9 // 4) - -9)))
  interpreter: [-7, 6, 45]
  compiler:    [-7, 6, 38]
```

One random program. The bug you might have shipped.

The oracle, extended.



Three things that should agree: a generator, a spec, and your compiler. You wrote one of the three.

What fuzzing can't do.

It finds *disagreements*, not *bugs* — if the interpreter is wrong too, both agree and nobody notices.

It finds only what the generator can reach. No loops in the generator → no loop bugs found.

It doesn't tell you *why*. That's still you.

So you still write tests by hand. Just different ones from the ones you'd have written before today.

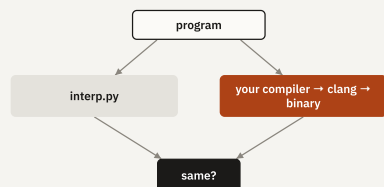
What to put in **tests/**.

Boundaries.	<code>print(2147483647 + 1)</code> — $2^{31} - 1$ is the largest int the manual allows (§2.5.1). What happens past it, the manual doesn't say — so the answer is whatever <i>your</i> interpreter does. Run it and find out; that is the whole method.
Sign combinations.	<code>//</code> and <code>%</code> with every pair of signs.
Control-flow shapes.	<code>if</code> with no <code>else</code> ; <code>while</code> that runs zero times; <code>if</code> inside <code>while</code> .
Things that must fail.	<code>print(1 // 0)</code> — whatever page 1 says it does, it must do <i>that</i> , and not crash.

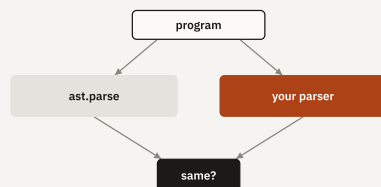
Your test suite is 2 of the 5 points on every PA. It is graded on *what it would catch*, not on how many tests it has — and what it would catch depends entirely on which box you are testing. That is the next three slides.

Four boxes. Four different answers to “compared against what?”

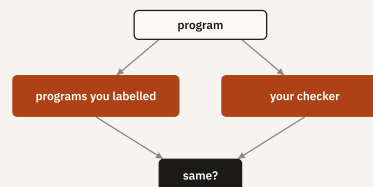
Code generator



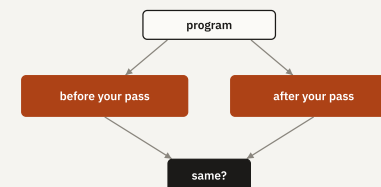
Lexer and parser



Type checker



Optimizer



Code generator	PA1	your binary's output ↔ <code>interp.py</code> 's output
Lexer and parser	PA2	your tree ↔ <code>ast.parse</code> 's tree
Type checker	PA3	your yes-or-no ↔ programs you labelled yourself
Optimizer	PA5–6	your program's output ↔ its output before your pass — one walk over the whole program that rewrites it; PA5 and PA6 are passes you write

One of these is handed to you and right. One is handed to you and wrong in a specific place. One does not exist until you build it. One compares you against yourself.

Every oracle you are given grades what you *accept*.

```
3 < 4 < 5      # Python parses it. ChocoPy has no chains (§4.1).
                # A correct parser REJECTS this – and ast.dump can't say so.

x: int = True   # Parses fine. Your checker must REJECT it.
                # interp.py runs it anyway and binds x = 1 – it grades what
                # programs DO, never whether they should have been allowed.
```

`ast.dump` prints a tree as text, so two trees can be compared with `==`. It is how PA2 asks whether yours is *the same tree*.

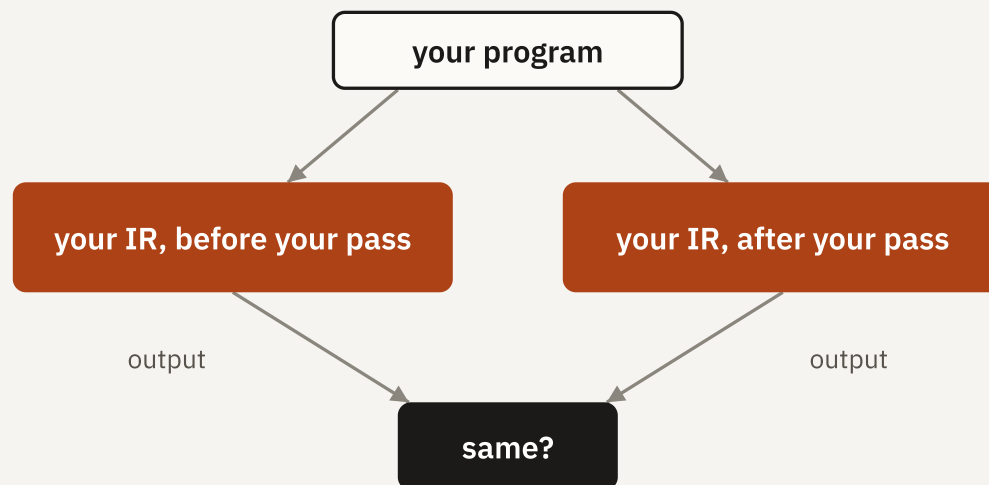
PA2's oracle is `ast.dump(yours) == ast.dump(ast.parse(src))`. Where ChocoPy and Python disagree, a *correct* rejection makes it report a failure — and where Python rejects a program outright, it never runs at all.

PA3 is worse: a type checker's whole job is saying **no**, and nothing in this course says no for you.

Both fail the same way. You can only be graded on the programs that get through.

So for these two you build the answer key yourself: a folder of programs you have decided are legal, a folder you have decided are not — and for each of the second kind, the rule that should catch it and the line it should name.

The optimizer's answer key is the program you started with.



PA5 and PA6 don't add behaviour. They rewrite the program and promise the answers don't move.

So the oracle is your own output twice: run it before your pass, run it after, compare.

Which means a bug in your lexer, parser or checker is invisible here — both runs are wrong in the same way, and they agree.

That is slide 19's problem again, one level up: two things that agree are only as good as the chance that they were going to be wrong *differently*.

Your spec is not your neighbour's spec.

The ChocoPy manual is the spec — and **page 1** of your PA1 handout overrides it on exactly two points: integer width (31 vs 32 bits) and what `x // 0` does (error, or result 0).

For PA1, use the behavior specified by the handout and the checks it describes.

The handout defines the supported subset and required behavior; the supplied `test/pa1/` suite and CPython differential harness are the operational check.

For PA1, ordinary output must match CPython, and division by zero must exit with code 2.

The half an AI model can't do for you.

An AI model will happily write `compile_expr`. It wrote `sdiv` for `//` too — that's where this bug came from.

It cannot tell you the assumption is wrong, because reading the code is not how you find out. *Running two things and comparing* is how you find out.

And it cannot know your twist. Nobody outside this room has your page 1.

Which is the honest reason the exams are on paper: the part of this that is yours is deciding what to test and how you'd know the answer.

This is how real compilers are tested.

Csmith (2011) generates random C programs; found 325 bugs in GCC and LLVM, most of them wrong-code bugs like yours.

Alive2 proves each LLVM optimization correct, or hands you the counterexample — week 9.

Every clang release tens of thousands of differential tests against previous releases.



You are doing the real thing at small scale. Same oracle, same failure mode, fewer zeros.